

JOB SYNK AI

AI Interviews Backend Handoff

Frontend implementation, required backend services, API contracts, database design, background processing, security, testing, and delivery checklist.

FastAPI

PostgreSQL

Audio uploads

Transcription

Evaluation

Reports

Prepared for: Backend engineering handoff

Repository: resume-ai-app

Date: 23 August 2026

Status: Frontend mock experience complete; live adapter pending backend

1. Executive summary

The AI Interviews frontend is a complete, protected, voice-first mock interview experience. A user creates a session, configures the target role, optionally attaches a stored resume and job description, checks the microphone, records one answer at a time, submits audio, waits for transcription/evaluation, and receives a detailed readiness report.

Frontend status: All screens, route guards, recorder controls, local mock persistence, deterministic questions, a contextual follow-up, answer evaluation, report rendering, retry UX, and history are implemented. The UI depends on an `InterviewApi` interface, so backend integration belongs in the live adapter rather than page components.

Backend status: No interview endpoints, tables, media storage, transcription jobs, evaluation jobs, or report generation exist yet. The mock implementation uses browser storage and simulated delays only.

Target backend outcome

Build an authoritative asynchronous interview service in the existing FastAPI backend. It must persist sessions across devices, safely accept audio, run transcription/evaluation outside request handlers, guarantee idempotent answer submission, enforce ownership/state transitions, and return stable view models to the frontend.

Current application architecture

Layer	Current technology	Interview responsibility
Frontend	React, TypeScript, Vite, React Router	Recording, playback, forms, polling, reports, navigation
Application API	FastAPI, SQLAlchemy, Alembic, PostgreSQL	Auth, ownership, session state, metadata, uploads, orchestration
AI service	Separate FastAPI service	Question generation, transcription adapter, evaluations, report synthesis
Media	Not selected	Object storage required for audio assets; DB stores metadata only
Jobs	Not currently present	Queue/worker required or a durable interim job mechanism

2. What the frontend currently implements

Routes and screens

Route	Purpose	Required server state
<code>/ai-interviews</code>	Introduction, readiness summary, completed count, history, continue/report/delete actions	List of non-deleted sessions
<code>/ai-interviews/new</code>	Role, type, seniority, duration, resume, job description, setup review	Resume options; creates a draft
<code>/:id/ready</code>	Instructions, consent, device selection, microphone test	<code>draft</code> or <code>ready</code>
<code>/:id/live</code>	Question, timers, record/pause/replay/re-record/submit, finish early	<code>in_progress</code>
<code>/:id/processing</code>	Final staged processing with refresh recovery	<code>processing</code>
<code>/:id/report</code>	Overall score, dimensions, strengths, improvements, plan, per-question feedback	<code>report_ready</code>

Frontend modules

File	Responsibility
<code>src/features/ai-interviews/page.tsx</code>	All route-level screens and navigation behavior
<code>components.tsx</code>	Recorder controls, score bars, report feedback, loading/error panels
<code>hooks.ts</code>	<code>useAudioRecorder</code> , microphone test, session loading
<code>types.ts</code>	UI domain types and <code>InterviewApi</code> interface
<code>utils.ts</code>	Validation, weighted score, time/date/status formatting, route destination
<code>api.ts</code>	<code>MockInterviewApi</code> and unimplemented <code>LiveInterviewApi</code>

Recording behavior

- Microphone access is requested only after a user action.
- Uses browser `MediaRecorder` ; maximum answer recording is 3 minutes.
- Device test recording is capped at 10 seconds.
- The user can pause, resume, stop, replay, and re-record.
- An empty `Blob` cannot be submitted.
- The audio `Blob` stays in browser memory until upload registration and answer submission succeed.
- A before-unload warning protects an unsent recording.
- Media tracks and object URLs are released on replacement/unmount.

3. Domain model and state machines

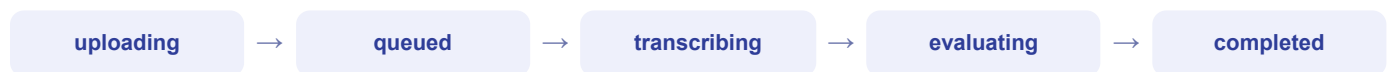
Session states



Terminal/exception states: `abandoned` , `failed` , `deleted` .

State	Meaning	Allowed actions
draft	Setup saved, preparation not finalized	Prepare, delete
ready	Questions/context prepared; device check may proceed	Start, delete
in_progress	Interview is accepting one authoritative current answer	Get question, register upload, submit answer, complete/abandon
processing	Final report jobs running	Get session/report status only
report_ready	Immutable report available	Get report, delete, practice again via new session
abandoned	User explicitly ended early	List/get/delete; no more answers
failed	Non-recoverable or exhausted processing failure	Retry if retryable, delete
deleted	Soft-deleted and hidden from normal list	None

Answer states



`recording` is primarily a client state. `failed` is a server state and must include a safe, user-facing error plus whether retry is allowed.

Server is authoritative. Reject invalid transitions with `409 state_conflict` . Never trust a client-provided session status, question index, score, transcript, evaluation, user ID, or media ownership.

Scoring contract

Dimension	Weight	Range
Relevance	30%	0-100
Evidence	25%	0-100
Structure	15%	0-100
Role alignment	20%	0-100
Communication	10%	0-100

```
overall_score = round(
    relevance * 0.30 + evidence * 0.25 + structure * 0.15
    + role_alignment * 0.20 + communication * 0.10
)
```

4. Required API surface

Base paths below assume the existing same-origin `/api` proxy. All endpoints require the current JWT bearer token and derive `user_id` from `get_current_user`.

Method and path	Purpose	Success
POST <code>/interviews</code>	Create draft and persist setup snapshot	201 session
POST <code>/interviews/{id}/prepare</code>	Build question plan from setup/resume/JD	200 ready session
GET <code>/interviews/{id}</code>	Authoritative session and recovery state	200 session
POST <code>/interviews/{id}/start</code>	Start or idempotently return in-progress session	200 session
GET <code>/interviews/{id}/current-question</code>	Return only current unanswered question and progress	200 question
POST <code>/interviews/{id}/uploads</code>	Create media asset and signed upload instructions	201 registration
POST <code>/interviews/{id}/answers</code>	Finalize asset, create answer idempotently, enqueue processing	202 answer
GET <code>/interviews/{id}/answers/{answer_id}</code>	Poll transcription/evaluation state	200 answer
POST <code>/interviews/{id}/complete</code>	Complete normally or abandon early	202/200 session
GET <code>/interviews/{id}/report</code>	Return report or processing status	200 report / 202 status
GET <code>/interviews</code>	Paginated history and summary fields	200 list
POST <code>/interviews/{id}/retry</code>	Retry a failed recoverable job	202 session/answer
DELETE <code>/interviews/{id}</code>	Soft delete session; schedule media deletion	204

Create interview

```
POST /interviews
{
  "interview_type": "behavioural",
  "role_title": "Frontend Developer",
  "seniority": "mid",
  "duration_minutes": 15,
  "resume_id": 12,
  "job_description": "Optional text, max 8000 chars",
  "voice_enabled": true
}
```

Validate enums, duration (10/15/20), role length, job-description length, and resume ownership. Snapshot relevant resume content at creation or preparation time so later resume edits do not silently change an active interview.

Prepare response

```
{
  "id": "int_uuid",
  "status": "ready",
  "estimated_question_count": 4,
  "questions": [
    {"id": "q_uuid", "prompt": "Tell me about...", "category": "behavioural", "is_follow_up": false, "position": 0}
  ],
  "current_question_index": 0,
  "follow_up_used": false,
  "updated_at": "2026-08-23T12:00:00Z"
}
```

5. Audio upload and answer submission

Recommended two-step upload

1. Frontend records a `Blob` and keeps it in memory.
2. `POST /interviews/{id}/uploads` registers expected media metadata.
3. Backend returns a short-lived signed object-storage upload URL.
4. Frontend uploads bytes directly to storage with required headers.
5. Frontend calls `POST /interviews/{id}/answers` with the registered media ID and stable idempotency key.
6. Backend verifies the media object exists, belongs to the user/session, is within limits, and is not already attached elsewhere.
7. Backend creates one answer transactionally and enqueues processing.
8. Only after success may the frontend release its recorded `Blob`.

Upload registration request/response

```
POST /interviews/{id}/uploads
{
  "content_type": "audio/webm;codecs=opus",
  "size_bytes": 845231,
  "duration_ms": 65000
}

201 Created
{
  "media_asset_id": "media_uuid",
  "upload_url": "https://storage.example/signed-url",
  "method": "PUT",
  "headers": {"Content-Type": "audio/webm;codecs=opus"},
  "expires_at": "2026-08-23T12:10:00Z",
  "max_size_bytes": 25000000
}
```

Answer submission

```
POST /interviews/{id}/answers
Idempotency-Key: answer_int_uuid_q_uuid_...
{
  "question_id": "q_uuid",
  "media_asset_id": "media_uuid",
  "duration_ms": 65000,
  "idempotency_key": "answer_int_uuid_q_uuid_..."
}

202 Accepted
{
  "id": "ans_uuid",
  "question_id": "q_uuid",
  "duration_ms": 65000,
  "status": "queued",
  "created_at": "2026-08-23T12:02:00Z",
  "poll_after_ms": 1000
}
```

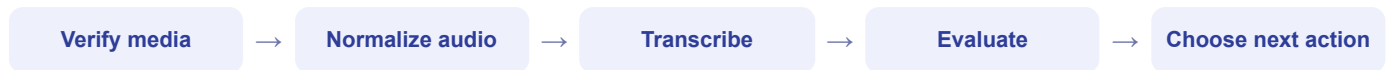
Idempotency requirement: Create a unique database constraint on `(user_id, idempotency_key)`. The same key with the same payload returns the original answer. The same key with a different payload returns `409 idempotency_conflict`. This must work across processes and restarts.

Limits and validation

- Accepted MIME types: browser formats actually supported by product testing (at minimum WebM/Opus; optionally MP4/AAC and Ogg/Opus).
- Maximum declared/actual recording duration: 180 seconds.
- Maximum object size: choose a safe value (recommend 25 MB) and enforce in signed upload policy and backend verification.
- Reject zero-byte objects, mismatched content types, expired registrations, wrong session/user, already-consumed assets, and malware/invalid media.
- Never proxy large audio through the main API unless object storage is unavailable for the first milestone.

6. Answer processing and next-question logic

Worker pipeline



1. Mark answer `queued` in the submission transaction.
2. Worker marks `transcribing`, downloads/streams the object, and calls the selected transcription provider.
3. Persist transcript text, provider IDs, language, duration, confidence where available, and sanitized failure details.
4. Mark `evaluating` and send question + transcript + role/JD/resume context to the AI service using a versioned structured-output schema.
5. Validate every score is 0-100 and all arrays/strings meet limits before saving.
6. Determine `next_action`: `follow_up`, `next_question`, or `complete`.
7. Transactionally update answer, current question position, optional follow-up question, and session timestamp.

Completed answer response

```
{
  "id": "ans_uuid",
  "question_id": "q_uuid",
  "status": "completed",
  "transcript": "In this example, I...",
  "evaluation": {
    "scores": {
      "relevance": 82,
      "evidence": 74,
      "structure": 79,
      "role_alignment": 81,
      "communication": 84
    },
    "evidence": "The answer identifies...",
    "worked": ["Clear ownership", "Specific actions"],
    "improvements": ["Quantify the result"],
    "improved_outline": ["Open with the outcome", "Give context", "Explain actions", "Close with result"]
  },
  "next_action": "follow_up",
  "next_question": {
    "id": "q_followup_uuid",
    "prompt": "What trade-off did you make?",
    "category": "contextual_follow_up",
    "is_follow_up": true,
    "parent_question_id": "q_uuid"
  }
}
```

Follow-up rule

The product allows a maximum of one contextual follow-up per interview. Enforce this server-side with `follow_up_used`, not only in the prompt. A follow-up should be generated only when it materially probes missing evidence or a useful trade-off; otherwise continue to the next planned question.

Polling now, SSE later

The live adapter should poll the answer endpoint with exponential/backoff-aware intervals (for example 1s, 1.5s, 2s; cap 3s) until completed/failed. Return `poll_after_ms`. Later, expose an authenticated SSE stream for status events without changing UI domain models.

7. Report generation contract

When the last answer completes, or the user normally completes the interview, transition the session to `processing` and enqueue a final report job. The report must be reproducible from stored answers/evaluations and include a schema/prompt version.

```
{
  "overall_score": 79,
  "scores": {
    "relevance": 82,
    "evidence": 73,
    "structure": 78,
    "role_alignment": 80,
    "communication": 84
  },
  "strengths": ["Answers stay relevant", "Clear decision-making"],
  "improvements": ["Add measurable outcomes", "Use a consistent STAR structure"],
  "action_plan": ["Prepare three achievement stories", "Practise 90-second answers"],
  "generated_at": "2026-08-23T12:10:00Z",
  "schema_version": "1.0",
  "evaluation_version": "interview-eval-v1"
}
```

Aggregation rules

- Dimension scores should be aggregated consistently across completed answers (document whether equal average or weighted by question type).
- Overall score must follow the frontend weights in section 3.
- Report generation should not invent factual evidence not present in transcripts or resume context.
- Store per-question evaluation separately; the report references it and must survive prompt/model upgrades.
- If one answer failed permanently, define whether normal completion is blocked or report is generated with a visible incomplete marker.

Current frontend assumption to change in LiveInterviewApi: the mock `submitAnswer` waits until evaluation completes. The real adapter should accept a 202 queued answer, poll `getAnswer`, then return the completed view model to the existing page flow.

8. Recommended database design

Table	Important columns
interview_sessions	id UUID, user_id FK, resume_id nullable, status enum/string, type, role_title, seniority, duration_minutes, job_description snapshot, resume_snapshot JSONB, current_question_index, follow_up_used, started_at, processing_started_at, completed_at, deleted_at, error_code, version, timestamps
interview_questions	id UUID, session_id FK, position, prompt, category, is_follow_up, parent_question_id nullable, generation_metadata JSONB, created_at
interview_media_assets	id UUID, user_id FK, session_id FK, storage_key, content_type, size_bytes, duration_ms, checksum, status (registered/uploaded/consumed/deleted), expires_at, created_at, consumed_at
interview_answers	id UUID, session_id FK, question_id FK, media_asset_id FK unique, user_id, status, idempotency_key, request_fingerprint, duration_ms, transcript, transcription_metadata JSONB, evaluation JSONB, next_action, attempt_count, error_code/message, timestamps
interview_reports	id UUID, session_id FK unique, overall_score, dimension_scores JSONB, strengths JSONB, improvements JSONB, action_plan JSONB, schema_version, evaluation_version, generated_at
interview_job_attempts (recommended)	id, entity_type/id, job_type, attempt, provider, provider_request_id, status, error_code, started_at, finished_at

Required constraints/indexes

- Foreign keys with deliberate delete behavior; do not cascade away audit-critical answer metadata accidentally.
- Unique (user_id, idempotency_key) on answers.
- Unique (session_id, position) on questions.
- Unique completed answer per (session_id, question_id), or a design that explicitly versions re-attempts.
- Unique report per session.
- Indexes on (user_id, updated_at desc), session status, answer status, registered asset expiry, and soft-delete fields.
- Optimistic version or row lock around current-question advancement to prevent double progression.

Migration/module placement

- app/models/interview.py
- app/schemas/interview_schema.py
- app/routers/interviews.py
- app/services/interviews/... for orchestration (avoid putting the workflow in router functions)
- New Alembic revision importing all interview models in migration/runtime model discovery
- Register router in app/main.py

9. AI service contracts

The application backend should remain the only service called by the browser. It may call the existing separate AI service using internal authentication/timeouts. Prefer new versioned, structured JSON endpoints rather than reusing text-stream endpoints designed for cover letters.

Suggested internal endpoint	Input	Output
POST /v1/interviews/questions	Role, type, seniority, duration, sanitized JD, resume snapshot	Ordered question plan with stable categories
POST /v1/interviews/evaluate-answer	Question, transcript, role context, prior answer summary, rubric	Validated dimension scores and feedback; optional follow-up intent
POST /v1/interviews/report	Session metadata and completed evaluations	Strengths, priorities, action plan; optionally dimension aggregation explanation

Transcription can be called from the application worker directly or behind the AI service. Whichever owns it must expose provider-neutral results and stable error categories.

Prompt and schema controls

- Use JSON-schema/structured output and validate before persistence.
- Version prompts, rubrics, and response schemas.
- Set deterministic/low-temperature behavior for scoring.
- Do not score accent, appearance, emotion, protected characteristics, or camera data.
- Never require video/camera; this product is asynchronous audio only.
- Apply timeouts, bounded retries, circuit breaking, and provider request IDs.
- Do not log raw resume/JD/transcript/audio in ordinary application logs.

10. Auth, security, privacy, and product policy

Authentication and authorization

- Use existing `get_current_user`; every query must include current user ownership.
- Return 404, not 403, for another user's interview/media ID to avoid resource enumeration.
- Resume IDs must be resolved using existing per-user resume semantics.
- Decide product gating before launch: if paid-only, apply existing `require_paid_plan` and add an `ai_interviews` usage feature consistently across config, events, UI, and documentation.

Privacy/retention decisions required

- Audio retention period (recommend short and explicit, e.g. automatic deletion after 30 days or immediately after transcription if replay history is not needed).
- Transcript/report retention and account-deletion behavior.
- User-visible consent language and privacy-policy update.
- Encryption in transit and at rest; private buckets only.
- Signed URL expiration and object key randomization; never expose permanent public URLs.
- Deletion job must remove storage object and mark/delete DB records according to policy.
- Redact secrets/PII from tracing and provider logs where possible.

Abuse/cost controls

- Rate-limit session creation, upload registration, answer submission, polling, and retries.
- Cap concurrent active sessions and queued jobs per user.
- Count billable work when a transcription/evaluation job is accepted, not for harmless GET polling.
- Validate declared size/duration against actual media metadata.
- Set global worker concurrency and provider budget alarms.

11. Error contract and recovery

```
{
  "detail": {
    "code": "state_conflict",
    "message": "This interview is not accepting answers.",
    "retryable": false,
    "current_status": "processing",
    "request_id": "req_uuid"
  }
}
```

HTTP	Code examples	Frontend behavior
400/422	validation_error, unsupported_media, empty_recording	Show field/recording guidance
401	authentication_required	Existing refresh/login flow
402	requires_plan	Existing upgrade modal, if gated
404	interview_not_found, answer_not_found	Recover to history with message
409	state_conflict, idempotency_conflict, question_already_answered	Refetch authoritative session
410	upload_expired	Register a new upload; retain blob
413/415	media_too_large, unsupported_media	Explain recording requirements
429	usage_limit_reached, too_many_requests	Respect Retry-After / existing modal where applicable
500/502/503	processing_failed, provider_unavailable	Keep recording if submission not accepted; otherwise poll/retry job

Retry semantics

- Upload URL expired before upload: register a new media asset.
- Upload succeeded, answer POST timed out: resend the exact same idempotency key/payload.
- Transcription/evaluation failed after answer accepted: do not re-upload; retry the server job.
- Client refresh during processing: `GET /interviews/{id}` must recover state.
- Worker crash: queued/in-progress jobs must be reclaimable using leases/timeouts.

12. Backend delivery plan

Milestone 1 - persistence and contract

- [] Add models, enums, Alembic migration, schemas, router registration.
- [] Implement create/list/get/delete with ownership and pagination.
- [] Implement prepare/start/current-question and transition guards.
- [] Provide OpenAPI examples matching this document.

Milestone 2 - media and idempotent answers

- [] Select S3-compatible storage and configure private bucket/CORS/lifecycle rules.
- [] Implement upload registration, object verification, checksums, limits, cleanup.
- [] Implement transactional idempotent answer creation.
- [] Add queue/worker and answer polling states.

Milestone 3 - intelligence pipeline

- [] Implement transcription provider adapter and normalization.
- [] Add structured question-generation/evaluation/report endpoints to AI service.
- [] Enforce one contextual follow-up.
- [] Persist versions/provider metadata without leaking sensitive content.

Milestone 4 - reliability and launch

- [] Implement retries, dead-letter handling, stalled-job recovery, metrics and alerts.
- [] Finalize plan gating, usage limits, retention policy, consent/privacy copy.
- [] Implement `LiveInterviewApi` DTO mapping and polling in frontend.
- [] Run full create-record-submit-process-report journey in staging.
- [] Load/cost test concurrent transcription/evaluation jobs.

13. Required tests and acceptance criteria

Backend automated tests

- Setup validation, resume ownership, enum/duration/JD limits.
- Every allowed and forbidden session transition.
- Concurrent start and concurrent answer submission.
- Same idempotency key/same payload returns same answer.
- Same idempotency key/different payload returns 409.
- Wrong user cannot read/delete/upload/submit against another session.
- Signed upload expiry, size/MIME mismatch, missing object, consumed media.
- Worker retry and stalled-job recovery.
- Maximum one contextual follow-up.
- Score validation and weighted overall calculation.
- Normal completion, early abandonment, final report, delete/media cleanup.
- Refresh/poll recovery at queued/transcribing/evaluating/processing states.

Definition of done

- [] A user can create an interview using a resume they own.
- [] Refresh at any route returns the user to the authoritative stage.
- [] An uploaded answer cannot be lost or double-billed after a network timeout.
- [] Audio is private, bounded, and deleted according to documented policy.
- [] Answer status is observable until completed or safely retryable.
- [] The next question/follow-up cannot advance twice under concurrency.
- [] The final report matches the required dimensions and weights.
- [] Frontend can switch to `VITE_INTERVIEW_API_MODE=live` without rewriting screens.
- [] Logs/metrics make provider failures and stuck jobs diagnosable.

14. Decisions the team must make before coding

Decision	Recommended starting point
Paid feature?	Use existing paid-plan gate; introduce <code>ai_interviews</code> usage key
Object storage	S3-compatible private bucket with signed PUT and lifecycle deletion
Job system	Durable Redis-backed worker (Celery/RQ/Arq) rather than FastAPI BackgroundTasks
Transcription provider	Provider adapter; choose based on supported browser formats, latency, cost, privacy
Audio retention	30 days maximum or delete after successful transcription unless playback history is required
Report with failed answer	Require retry before normal completion; allow explicit early-abandon without report
Question count	10 min = 3, 15 min = 4, 20 min = 5, plus max one follow-up
SSE timing	Launch with polling; add SSE after contract is stable
Re-record after submit?	No for v1; re-record is allowed only before accepted submission

Frontend integration touchpoint

The backend developer should provide a staging base URL/OpenAPI schema and example credentials. Frontend work then stays focused inside `LiveInterviewApi` : snake_case DTO mapping, signed upload execution, polling, auth wrapper use, and normalized errors. Page and component code should not import backend DTOs directly.

Recommended handoff order: agree on state transitions and JSON schemas first; implement database/ownership second; prove idempotent media submission third; integrate AI providers only after the durable workflow is passing tests.

Source of truth used for this document: the implemented React AI Interviews feature and current FastAPI repository structure as of 23 August 2026. This is an engineering handoff, not a claim that the backend endpoints already exist.